ENGINEERING DOCUMENTATION

Technical Documentation

Architecture, AI safety, API contracts, operations and production delivery reference.

VERSION 1.0

LUMINA COMMERCE, INC.

ENGINEERING REFERENCE

System overview

Lumina is a full-stack commerce application built with React, TypeScript, FastAPI, PostgreSQL, pgvector, LangChain and OpenAI. The same FastAPI deployment serves the compiled storefront and versioned JSON APIs. Vercel hosts production and GitHub Actions validates every change.

The assistant is retrieval-first. The model may write an answer only after trusted chunks pass retrieval and relevance checks, and every live answer must cite valid source identifiers.

Core stack

Layer	Technology	Responsibility
Web	React, TypeScript, Vite	Storefront, localization, cart, checkout and Lumi UI.
API	Python, FastAPI, Pydantic	Typed HTTP contracts, middleware, orchestration and static delivery.
AI	LangChain, ChatOpenAI	Structured model calls, prompt controls and fallback handling.
Retrieval	PostgreSQL FTS, pgvector, local KB	Hybrid candidates, fusion, reranking and citations.
Data	SQLAlchemy async, PostgreSQL	Orders, products, document chunks and vector storage.
Delivery	Vercel, GitHub Actions	Production deployments and continuous quality checks.

REQUEST AND DEPLOYMENT TOPOLOGY

Runtime architecture

Browser request flow

Stage	Behavior
1. Edge routing	The Vercel production alias routes the request to the Lumina project.
2. FastAPI entrypoint	backend.app.main:app initializes middleware, routers and the application lifespan.
3. API or asset	Requests under /api/v1 and /health use API routers; other paths are served from frontend/dist.
4. Response security	Security headers, request identifiers, compression and rate controls are applied.
5. Client state	React manages locale, product filtering, cart state, checkout state and assistant history.

Repository map

- frontend/src - React components, catalog fallback, i18n, API client and styles.
- frontend/public/assets - locally served product photography and Lumina brand assets.
- backend/app/api/routes - health, catalog, order and assistant endpoints.
- backend/app/services - commerce and assistant application services.
- backend/app/rag - ingestion, guardrails, local knowledge, retrieval and result types.
- backend/knowledge - approved company documents used by retrieval.
- backend/evals - golden dataset and RAGAS evaluation entrypoint.
- .github/workflows - continuous integration pipeline.

GROUNDED RESPONSE GENERATION

RAG and LLM pipeline

1. Validate the request

Pydantic validates locale, session identifier, message length, history and cart context. Rate limiting applies specifically to chat traffic.

2. Inspect prompt injection risk

PromptInjectionGuard detects direct instruction override attempts, prompt disclosure requests, role markup and common obfuscation patterns. High-risk requests receive a safe refusal before retrieval or model invocation.

3. Classify intent and build a retrieval query

The service classifies recommendation, shipping, returns, warranty, payment and product intents. Multilingual queries receive stable English retrieval anchors for consistent lexical behavior.

4. Retrieve and rerank

When PostgreSQL is available, the retriever combines lexical full-text candidates and pgvector semantic candidates using reciprocal rank fusion. Deterministic keyword and source signals rerank the fused set. A local knowledge retriever provides graceful degradation.

5. Enforce evidence thresholds

Chunks below the minimum relevance score are removed. If no strong evidence remains, Lumi abstains without calling the model.

6. Generate a structured answer

ChatOpenAI receives a system policy, sanitized conversation context, cart summary, current question and delimited trusted sources. The response must match the LLMAnswer JSON schema.

7. Verify the result

The service rejects unanswerable, low-confidence or uncited outputs. Cited identifiers must exist in the retrieved set. Any model exception falls back to a controlled local answer.

DEFENSE IN DEPTH

AI safety controls

Control	Implementation	Failure behavior
Source isolation	Retrieved text is wrapped as source data and explicitly cannot supply instructions.	Ignore embedded commands.
Prompt injection guard	Rules inspect override, exfiltration, role and encoded attack patterns.	Return safe-refusal mode.
Low temperature	LLM temperature is 0.1 by default.	Reduce creative variance.
Structured output	Pydantic-backed JSON schema defines answerability, confidence, citations and follow-ups.	Reject invalid output.
Citation validation	Only identifiers present in retrieved chunks are accepted.	Abstain if no valid citation remains.
Relevance gate	Minimum chunk score defaults to 0.32.	Do not call the model without evidence.
PII guidance	System policy prohibits passwords, tokens, documents and secrets.	Redirect to safe support.
Operational fallback	Model and database failures have controlled local paths.	Preserve availability with reduced capability.

Security headers

- Content Security Policy permits same-origin scripts, images and API calls while blocking objects and framing.
- X-Content-Type-Options prevents MIME sniffing.
- X-Frame-Options and frame-ancestors prevent clickjacking.
- Permissions-Policy disables camera, microphone and geolocation access.
- Referrer-Policy limits cross-origin referrer exposure.

VERSIONED HTTP INTERFACE

API contracts

Method	Path	Purpose
GET	/health	Runtime health, database mode, assistant mode and application version.
GET	/metrics	Prometheus-compatible runtime metrics when enabled by the service.
GET	/api/v1/products	Localized catalog source for storefront products.
POST	/api/v1/orders/checkout	Validate order input and create a simulated paid order.
POST	/api/v1/assistant/chat	Retrieve evidence and return a grounded assistant response.

Assistant response fields

- message - customer-facing answer in the requested locale.
- intent - classified commerce intent.
- confidence - bounded evidence and model confidence.
- grounded - whether the answer is supported by retrieved evidence.
- mode - live-rag, local-rag, safe-refusal or abstention behavior.
- citations - validated source title, origin and excerpt.
- suggestions - up to three guided follow-up questions.
- actions - validated UI actions such as opening a product or cart.

PRODUCTION OPERATIONS

Configuration and secrets

Variable	Required	Purpose
APP_ENV	Yes	Enables production behavior and disables interactive API docs.
APP_SECRET_KEY	Yes	Application secret used for production-safe configuration.
DATABASE_URL	For live pgvector	Async PostgreSQL connection string with the vector extension.
OPENAI_API_KEY	For live LLM	Server-side OpenAI credential stored as a Vercel secret.
LLM_MODEL	Optional	Defaults to gpt-4.1-mini.
EMBEDDING_MODEL	Optional	Defaults to text-embedding-3-small.
LLM_TEMPERATURE	Optional	Defaults to 0.1.
FRONTEND_ORIGIN	Optional	Allowed browser origin for CORS.

Secret handling

- Never commit .env or production secrets.
- Use Vercel encrypted environment variables for production credentials.
- Redeploy after changing a production environment variable.
- Rotate a credential immediately if it appears in logs, commits or client bundles.
- Only the backend can read OPENAI_API_KEY; the React build never receives it.

CI/CD, TESTS AND OBSERVABILITY

Delivery and quality

Continuous delivery

The GitHub repository is connected to Vercel. A push to main creates a production deployment; pull requests receive preview deployments. The commercial production domain follows the latest successful production deployment.

Continuous integration

- Ruff validates Python style and common correctness rules.
- Pytest covers commerce behavior, retrieval knowledge and prompt-injection controls.
- TypeScript compilation validates client contracts.
- Vite builds the production client bundle.
- A CSP regression test keeps storefront assets executable under the security policy.

Evaluation

The golden dataset contains representative, adversarial and unanswerable questions. The optional RAGAS workflow can measure faithfulness, answer relevance and context quality before model, prompt or retriever changes are released.

Observability roadmap

- Export OpenTelemetry traces for request, retrieval and model spans.
- Track p50 and p95 latency, token cost, fallback rate, abstention rate and citation validity.
- Replace in-memory rate limiting with Redis for multi-instance consistency.
- Process ingestion and blue-green index updates through a durable queue.

RELEASE AND VERIFICATION

Deployment runbook

Standard release

- Run Python linting and the backend test suite.
- Run the production frontend build.
- Review staged changes and confirm no secrets are tracked.
- Push the approved commit to main.
- Wait for GitHub Actions and the Vercel production deployment to succeed.
- Verify the production home page, /health, product API and assistant response mode.
- Perform a visual smoke test in desktop and mobile layouts.

Assistant verification

- Ask a supported product comparison and require grounded=true.
- Confirm mode=live-rag when OPENAI_API_KEY is configured.
- Confirm at least one citation supports the factual answer.
- Send a prompt-injection attempt and require safe-refusal mode.
- Ask an unsupported question and confirm abstention rather than fabrication.

Rollback

If a release fails smoke testing, promote the previous Ready deployment in Vercel and investigate in a branch. Do not remove the stable production alias during rollback.

Production endpoint

https://lumina-smart-store.vercel.app